

KROVIN, 让每一次「AI 替人动手」都先给人看明白

跑在 NVIDIA DGX Spark 上的开发流 Agent：读 Linear issue、Obsidian 纪要和人的口述，完成拆解、计划、开发、日志与状态同步。

参赛方向 DGX Spark 黑客松

日期 2026-09-29

GitHub github.com/arvelvale/krovin

一、项目说明

KROVIN 是一个部署在 NVIDIA DGX Spark 节点上的开发流 Agent。它读入 Linear issue、Obsidian 会议纪要和用户口述，跑通「拆解 → 计划 → 开发 → 写日志 → 同步状态」的完整链路：把大 issue 拆成带验收标准的子任务并征得确认后写回 Linear，写出实施计划，在独立分支上改代码、跑测试，按仓库约定补开发日志，最后同步状态。整个过程留在同一个对话里，每一步决策都有迹可循。

核心亮点一：一切「选哪个」交给独立的决策层。JEV (TypeSafe System One 模型) 负责技能选择、模型路由、工具门控、记忆精选与上下文压缩——它只做选择题和评分，不替主模型生成答案；每次选择的概率分布、置信度与所过阈值都写进决策轨迹，面板右侧可逐条翻阅、可复算。

核心亮点二：人掌握方向有具体的按钮。写操作默认弹出确认卡，先用一句人话说清「要做什么」和「JEV 怎么看」，原始参数折叠在下；被拒绝的调用完整留痕，Agent 不会绕路重写。运行中还可随时点停止键，内核在步边界收尾，未完成事项留在工作记忆里。

核心亮点三：模型按难度与隐私路由。本地 Nemotron 承担主力推理；连续非法工具参数或难题自动升级到 StepFun；隐私记忆只在私有模型可见，外部调用按深度过滤。

技术实现与架构：内核为单进程 Python，主循环按步推进，每步前检查上下文压缩与停止请求；技能以 `skills/*/SKILL.md` 交付，frontmatter 带触发词、工具白名单与负向路由，技能索引每轮只带一行、命中后才读全文；工具层覆盖文件、git、白名单命令、OpenShell 沙箱、Linear 与 Obsidian；上下文超预算时按块压缩，保护目标、约束与未完成事项，原始记录落盘可恢复；Web 面板把会话、模型状态、确认动作与决策轨迹收在同一块工作台。

优化方案：token 估算倍率用真实用量在线校准；压缩预算与全部阈值集中在 `agent/config.py` 并带方向注释，改动必须跑对应评估；A/B harness 对同一任务集比较「有 / 无 JEV 决策层」两臂。当前数据如实标注：技能选择任务集为自建训练集，26 条调参后 JEV 臂 100% 对主模型自选 88.5%；扩到 60 条后为 81.7% 对 75.0%，标定仍在进行——100% 是训练集成绩，不代表新任务表现。

二、部署说明

2.1 利用本地算力如何部署智能体

智能体主体部署在 DGX Spark 节点 (GB10, 121 GiB 统一内存)：内核与面板以单进程常驻节点，模型推理全部在本地完成——Nemotron 30B-A3B 由 vLLM 服务，Qwen3.8 27B 由 Ollama 服务。使用方按以下步骤操作：

- 1 首次部署（一次性）：**双击 `krovin-panel.bat`，把代码同步到节点并启动常驻面板（tmux 会话 `krovin`，只听 127.0.0.1，不上公网）。
- 2 每次使用：先启动 SSH 中转。**双击 `D:\bigproject\DGX-Spark\krovin-tunnel.bat` 并保持窗口打开——它建立本机到节点的反向隧道，做两件事：把节点面板转发到本机 127.0.0.1:9000；让节点访问 JEV / Linear 的境外流量借道本机代理（mihomo，默认 127.0.0.1:10090）出境。隧道断开自动重连。
- 3 再打开面板：**浏览器访问 `http://127.0.0.1:9000`，输入团队访问口令，进入落地页——可下载本报告书，或点「进入工作台」。
- 4 填写自己的配置：**进入工作台后，在左栏「集成设置」填自己的 Linear API Key（保存前真实连接一次）与 Git 提交身份；在「模型设置」可按需登记任意 OpenAI 兼容供应商。密钥只存节点（600 权限），接口不回传原文。
- 5 开始干活：**新建对话、选择工作区（节点上的一份副本：克隆 git 仓库、上传本地文件夹或新建空白项目），即可下达任务。

如不使用本机中转

JEV 与 Linear 将不可用：技能选择退回主模型自选，写操作全部改为人工确认；聊天、改代码、跑测试不受影响。关上中转窗口不影响面板常驻，重开即恢复。节点出不了境外是环境事实，我们用 SSH 反向隧道借道操作机代理解决，代价是操作机必须在线——如实告知，不包装成「全离线」。

2.2 如何优化大模型

- 推理服务：**vLLM 0.28.0（含 Triton 3.7.1、flashinfer 0.6.16）服务 Nemotron-3.5-Lightning-30B-A3B-NVFP4，量化权重约 21 GB，全量驻留统一内存；两轮代码任务实测观察约 80 token/s（样本极少，仅作观察记录，不作质量结论）。
- 备用兜底：**Ollama + Qwen3.8 27B（GGUF Q4_K_M，17 GB）作 backup 档，本地主力不可用时自动降级，重试与超时集中在端点配置。

- **运行时**：本地主循环可关思考（`AGENT_LOCAL_THINKING=0`）；token 估算按真实 prompt_tokens 在线校准倍率；上下文超预算即按块压缩，保护目标、约束、未完成事项与证据引用。
- **路由**：JEV 按任务难度决策模型档位，难题升级 StepFun step-5-preview；本地模型连续给出非法工具参数也触发升级。
- **评估过未采用（如实标注）**：NIM 容器（节点无法直连 nvcr.io、无 NGC key，官方 playbook 步骤已定位）与 TensorRT-LLM 编译路线（GGUF 权重需转换格式），后续若网络与凭据到位再验证。

2.3 如何设计 Agent Skills

设计出发点：技能不是能力大杂烩，而是开发流的工序。KROVIN 的技能围绕「读 issue / 纪要 → 拆解 → 计划 → 开发 → 审查 → 日志 → 状态同步」这条真实流水线配对，每个技能只对一段工序负责；工序之间的边界不靠模型自觉，靠格式契约和负向路由约束。

工序	技能	干什么	关键约束（写进 frontmatter）
拆	issue-breakdown	把 Linear issue 或一段需求拆成可独立完成的子任务，每条带验收标准	经用户确认后才作为子 issue 写回 Linear
拆	meeting-to-tasks	从 Obsidian 会议纪要提取行动项，查重后建 issue 草稿	经用户确认后才写入 Linear
计	plan-writer	为选定 issue 写实施计划：切片、验收、风险、涉及文件，存 docs/plans/	只写计划文档，不动代码
开	implement-change	按 issue 或计划改代码、补测试并跑通，在独立分支上提交	白名单仅限文件、git、白名单命令与沙箱
开	build-webpage	从零写能在浏览器直接运行的静态网页 / 小游戏（可用 CDN 版 React、Vue）	产物限定工作区内，完成后浏览器验收
审	review-gate	对照工作区硬约定审查改动，按「阻断 / 建议 / 通过」分级给结论	只读，不改任何内容
记	progress-logger	把当天提交与完成的 issue 整理成 docs/progress/<日期>.md 留档	只写文件、不改 Linear 状态；固定本地模型
记	standup-brief	生成「昨日完成 / 今日计划 / 阻塞」三段口头简报	只在对话里回复；固定本地模型

同	status-sync	按提交记录与测试结果把 Linear 状态和进展评论同步到最新	只改 Linear、不写日志；固定本地模型
---	-------------	---------------------------------	-----------------------

另有 3 个 internal 技能不进入选择索引，供流程内引用：branch-hygiene（分支命名、未提交改动、与 main 同步状态）、changelog-sync（按版本归类生成 CHANGELOG）、test-gap-finder（diff 测试缺口分析）。

- **物理形态与加载校验：** `skills/<name>/SKILL.md`，frontmatter 含 name、description（含触发词的职责描述）、argument-hint、allowed-tools（工具白名单）、not-for（负向路由）、model（auto / local / cloud）、version，还可声明自带脚本（read 直接跑、write_local 走 JEV 门控）。加载即校验：name 必须 kebab-case 且等于目录名、description ≤ 300 字、version 形如 x.y.z、触发词 ≥ 2 条、not-for ≥ 1 条、白名单工具必须已注册、声明的脚本必须真实存在——任一不过就只跳过该技能并记录错误，不影响其它技能上线。
- **特点一：只读与写入分级，白名单随命中技能生效。** 纯读取技能（standup-brief、review-gate）不带写工具；写入类也只拿得到自己声明的工具，主循环按「本轮命中技能的 allowed-tools 并集」放行写操作，未声明的工具直接拒绝。跑命令一律进 OpenShell 沙箱：非 root、系统目录只读、网络只放行软件源，策略 YAML 版本化随仓库走。例外是 Bypass 模式：人工接管时跳过白名单与逐次确认，但开发命令仍强制走沙箱。
- **特点二：按开销路由模型。** 轻量整理类（progress-logger、standup-brief、status-sync）标 `model: local`，固定走本地模型、低延迟低成本；其余技能交给路由层按任务难度决定本地或云端。
- **特点三：not-for 负向路由，专门拆近义词。** 每个技能至少声明一条「这不是我该干的，该去 X」。最容易混的三兄弟：status-sync（改 Linear）、progress-logger（写日志）、standup-brief（口头汇报）互相指向；test-gap-finder 也写明「补写缺失的测试 → implement-change」「review 测试质量 → review-gate」。第一级选错时，第二级精排还有机会纠回来。
- **一轮请求的工作流程：** ① 系统提示常驻技能索引，每个技能只占一行（名称 + 描述），命中前全文不进上下文；② 第一级 JEV Choice 对全部技能宽排（含「都不用」选项），同时问三个门控问题：要对着仓库 / 任务系统 / 笔记做具体的事吗、资深工程师会按团队既定流程或模板处理吗、纯凭通用知识文字回答就够吗（反向计分）；门控值（三问均值）< 0.30 或 P(none) ≥ 0.50，本轮就不加载任何技能，闲聊和概念问答在这一步被挡掉；③ 第二级只对 top3 候选加载正文开头各 700 字，逐个问 fits——只看职责是否直接覆盖请求的主要目标；最高 fits < 0.30 全部拒绝，其余 fits ≥ 0.50 的一起加载，天然支持「拆解 + 计划」这类组合技能；④ 命中后全文进上下文，写操作确认卡照常弹出。JEV 不可用时自动降级为主模型自选（看同一份清单输出 JSON），这条降级路径同时就是 A/B 评估的对照臂；JEV 结果缓存命中时耗时近零，面板据此标注。
- **质量保障与当前数据：** 评测任务集分「有技能覆盖 / 无覆盖 / 需组合」三类，含近义技能对与负向用例；A/B 对照「JEV 选择 vs 主模型自选」，输出正确率、误选率、token 与耗时。当前数据如

实标注：技能选择任务集为自建训练集，26 条调参期 JEV 臂 100% 对主模型自选 88.5%；扩到 60 条后为 81.7% 对 75.0%，标定仍在进行——训练集成绩不代表新任务表现。

- **接口冻结：**技能文件、任务集、决策轨迹三份格式契约冻结（只增不改），多人并行开发不改字段，评测脚本与实现解耦。

三、技术栈说明

类别	组件	用途
NVIDIA 平台与 SDK	NVIDIA DGX Spark (GB10 · aarch64 · 统一内存 121 GiB)	全部本地推理与智能体运行的硬件平台
	CUDA (驱动 580.142, 报告 CUDA 13.0)	GPU 计算底座; vLLM / Ollama 经其调用 GPU
	vLLM 0.28.0 (Triton 3.7.1 · flashinfer 0.6.16)	本地主力推理服务 (OpenAI 兼容接口)
	NVIDIA Nemotron-3.5-Lightning-30B-A3B-NVFP4	本地主力模型 (NVFP4 量化, 约 21 GB)
	NVIDIA OpenShell	Agent 工具执行沙箱 (内核级隔离 + YAML 策略)
	NVIDIA Nsight Systems (nsys) / DGX Dashboard	节点预置的性能剖析与运维面板
模型	Ollama 0.33.2 + Qwen3.8 27B (GGUF Q4_K_M)	备用模型服务 (已验证可用)
	StepFun step-5-preview	云端难题档模型 (复杂规划、代码难题)
决策层	JEV (TypeSafe System One, jev-latest)	结构化决策: 技能选择 / 模型路由 / 工具门控 / 记忆精选 / 上下文压缩
工程栈	Python 3.12 · TypeScript + Vite · SQLite · tmux · SSH 反向隧道	单体内核 + 自建 HTTP/SSE 面板; 记忆与设置落库; 长任务 tmux 托管; 跨境流量经操作机代理中转

技能共 12 个 (9 个对外可选 + 3 个内部, 覆盖 dev-flow 全链路 with 网页构建), 全部以 markdown 交付于仓库 `skills/` 目录; 决策轨迹、任务集格式契约见 `docs/接口/`。